

# RENFORCEMENT POO

Cours de préparation — S7 M1 Informatique (ILIA)

---

Approfondissement de la programmation orientée objet en Java : SOLID, design patterns, généricité avancée, interfaces modernes, gestion d'erreurs et immutabilité — avec exercices corrigés.

## Sommaire

1. Rappels ciblés : encapsulation, héritage, polymorphisme
2. Principes SOLID
3. Design patterns fondamentaux
4. Généricité avancée
5. Interfaces avancées
6. Gestion d'erreurs et immutabilité
7. Exercices récapitulatifs

# Chapitre 1 — Rappels ciblés

Tu as déjà travaillé l'encapsulation, `this`` et `@Override`` — ce chapitre condense juste ce qu'il faut avoir en tête avant d'aller plus loin, sans repartir de zéro.

## 1.1 Les trois piliers, en une phrase chacun

| Pilier        | idée clé                                                                                                            |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Encapsulation | Cacher l'état interne (champs privés) derrière une interface publique contrôlée (getters/setters, méthodes métier). |
| Héritage      | Une classe fille réutilise et spécialise le comportement d'une classe mère (relation « est-un »).                   |
| Polymorphisme | Un même appel de méthode déclenche un comportement différent selon le type réel de l'objet à l'exécution.           |

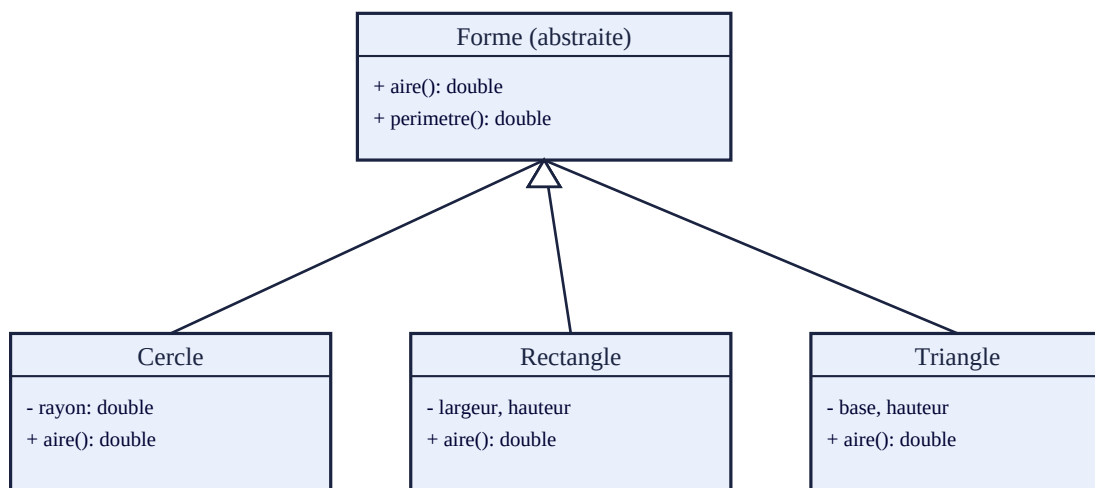


Figure 1.1 — Hiérarchie *Forme (abstraite)* → *Cercle*, *Rectangle*, *Triangle*

En Java, ``Forme`` serait déclarée `abstract`, avec une méthode `abstract double aire()` : chaque sous-classe est **obligée** de fournir sa propre implémentation, ce qui garantit le polymorphisme à l'appel.

**Piège classique :** confondre *surcharge* (overloading, même nom de méthode, signatures différentes, résolu à la compilation) et *redéfinition* (overriding, `@Override``, résolu à l'exécution grâce au polymorphisme).

# Chapitre 2 — Principes SOLID

SOLID est un acronyme regroupant cinq principes de conception orientée objet, destinés à produire du code plus maintenable et plus facile à faire évoluer.

| Lettr<br>e | Principe              | En bref                                                                        |
|------------|-----------------------|--------------------------------------------------------------------------------|
| S          | Single Responsibility | Une classe = une seule raison de changer                                       |
| O          | Open/Closed           | Ouvert à l'extension, fermé à la modification                                  |
| L          | Liskov Substitution   | Une sous-classe doit pouvoir remplacer sa classe mère sans casser le programme |
| I          | Interface Segregation | Préférer plusieurs interfaces spécifiques à une seule interface énorme         |
| D          | Dependency Inversion  | Dépendre d'abstractions (interfaces), pas d'implémentations concrètes          |

## 2.1 Illustration : violation du principe Open/Closed

Code à éviter (nécessite de modifier la classe existante à chaque nouvelle forme) :

```
class CalculateurAire {
    double calculer(Object forme) {
        if (forme instanceof Cercle) { ... }
        else if (forme instanceof Rectangle) { ... }
        // il faut modifier cette classe à chaque nouvelle forme !
    }
}
```

Version conforme (chaque forme fournit sa propre méthode aire()) :

```
abstract class Forme {
    abstract double aire();
}
class Cercle extends Forme {
    double aire() { return Math.PI * rayon * rayon; }
}
// Ajouter une nouvelle forme n'oblige à modifier aucune classe existante
```

### Exercice 1 — Identifier une violation SOLID

Une classe GestionnaireUtilisateur s'occupe à la fois de valider les données d'un utilisateur, de les enregistrer en base de données, et d'envoyer un email de confirmation. Quel principe SOLID est violé ? Comment le corriger ?

### Correction

C'est une violation du **principe de responsabilité unique (S)** : la classe a trois raisons de changer (règles de validation, format de stockage, service d'email). On corrige en séparant en trois classes distinctes : `ValideurUtilisateur`, `RepositoryUtilisateur` et `ServiceEmail`, chacune injectée là où elle est nécessaire.

# Chapitre 3 — Design patterns fondamentaux

Un **design pattern** est une solution réutilisable à un problème de conception récurrent. En voici quatre parmi les plus utilisés.

## 3.1 Singleton

Garantit qu'une classe n'a qu'une seule instance, accessible globalement.

```
public class Configuration {
    private static Configuration instance;
    private Configuration() {}
    public static Configuration getInstance() {
        if (instance == null) instance = new Configuration();
        return instance;
    }
}
```

## 3.2 Factory (fabrique)

Délègue la création d'objets à une méthode dédiée, plutôt que d'appeler `new` directement — utile quand le type exact à créer dépend d'un paramètre.

```
static Forme creerForme(String type) {
    return switch (type) {
        case "cercle" -> new Cercle();
        case "rectangle" -> new Rectangle();
        default -> throw new IllegalArgumentException(type);
    };
}
```

Note l'usage du switch expression moderne — cohérent avec ce que tu as déjà pratiqué.

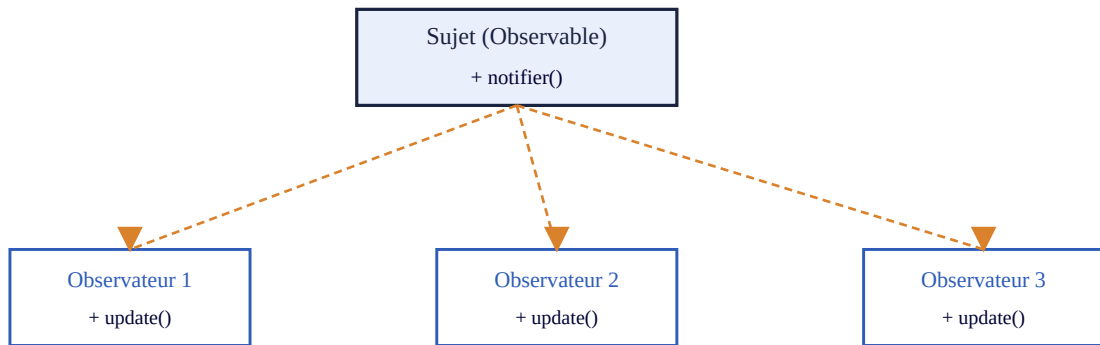
## 3.3 Strategy

Encapsule une famille d'algorithmes interchangeables derrière une interface commune, pour changer de comportement à l'exécution sans modifier la classe qui l'utilise.

```
interface StrategieTri { void trier(int[] tab); }
class TriRapide implements StrategieTri { public void trier(int[] tab) { ... } }
class TriFusion implements StrategieTri { public void trier(int[] tab) { ... } }
```

## 3.4 Observer

Permet à un ensemble d'objets (observateurs) d'être notifiés automatiquement quand l'état d'un autre objet (sujet) change — base des architectures événementielles.



notifier() appelle update() sur chaque observateur enregistré

Figure 3.1 — Pattern Observer : le sujet notifie tous ses observateurs enregistrés

## Exercice 2 — Choisir le bon pattern

Pour chaque situation, indiquez le pattern le plus adapté (Singleton, Factory, Strategy, Observer) :

- a) Une application météo doit mettre à jour plusieurs widgets d'affichage dès que la température change.
- b) Un jeu propose trois modes de difficulté avec des comportements d'IA différents, sélectionnables en cours de partie.
- c) Une application ne doit avoir qu'un seul gestionnaire de connexion à la base de données.

### Correction

- a) Observer — les widgets s'abonnent aux changements de température.
- b) Strategy — chaque mode de difficulté est une implémentation interchangeable d'une interface commune de comportement d'IA.
- c) Singleton — une seule instance du gestionnaire de connexion doit exister.

## Chapitre 4 — Généricité avancée

La généricité permet d'écrire du code réutilisable pour différents types tout en gardant la sécurité de typage à la compilation. Au-delà de `List<T>`, Java propose des mécanismes plus fins pour borner ces types.

### 4.1 Bornes génériques (bounded types)

```
class Boite<T extends Comparable<T>> {
    private T contenu;
    boolean estPlusGrandQue(Boite<T> autre) {
        return contenu.compareTo(autre.contenu) > 0;
    }
}
```

Ici, `T extends Comparable<T>` garantit que `T` possède bien une méthode `compareTo`, ce qui n'est pas garanti par un type générique non borné.

### 4.2 Wildcards : ? extends vs ? super

| Notation    | Signifie                                                 | Usage typique                                                                            |
|-------------|----------------------------------------------------------|------------------------------------------------------------------------------------------|
| ? extends T | Un type T ou l'un de ses sous-types (lecture seule sûre) | Méthode qui consomme/lit une collection : <code>List&lt;? extends Number&gt;</code>      |
| ? super T   | Un type T ou l'un de ses super-types (écriture sûre)     | Méthode qui produit/écrit dans une collection : <code>List&lt;? super Integer&gt;</code> |

**Principe PECS** (Producer Extends, Consumer Super) : utilisez `extends` quand la structure *produit* (vous lisez dedans), et `super` quand elle *consomme* (vous écrivez dedans).

#### Exercice 3 — Wildcards génériques

Vous écrivez une méthode qui copie tous les éléments d'une liste source vers une liste destination : `void copier(List<?> source, List<?> dest)`. Quels wildcards utiliser sur `source` et `dest`, en appliquant le principe PECS ?

##### Correction

`source` ne fait que produire des éléments (on les lit) : `List<? extends T> source`. `dest` ne fait que consommer des éléments (on y écrit) : `List<? super T> dest`. Signature finale : `<T> void copier(List<? extends T> source, List<? super T> dest)`.

# Chapitre 5 — Interfaces avancées

Depuis Java 8, les interfaces ne se limitent plus à des méthodes abstraites : elles peuvent fournir du comportement concret, ce qui a profondément changé leur usage.

## 5.1 Méthodes par défaut et statiques

```
interface Vehicule {  
    void demarrer();  
    default void klaxonner() { System.out.println("Beep!"); }  
    static Vehicule voiture() { return () -> System.out.println("Vroum"); }  
}
```

Les méthodes `default` permettent d'ajouter du comportement à une interface sans casser les classes qui l'implémentaient déjà (rétrocompatibilité).

## 5.2 Interfaces fonctionnelles et lambdas

Une **interface fonctionnelle** ne déclare qu'une seule méthode abstraite, ce qui permet de l'implémenter avec une expression lambda plutôt qu'une classe entière.

```
@FunctionalInterface  
interface Operation { int appliquer(int a, int b); }  
  
Operation addition = (a, b) -> a + b;  
System.out.println(addition.appliquer(3, 4)); // 7
```

**À connaître** : les interfaces fonctionnelles standard de `java.util.function` — `Function<T,R>`, `Predicate<T>`, `Supplier<T>`, `Consumer<T>` — évitent de redéfinir sa propre interface à chaque fois.

### Exercice 4 — Interface fonctionnelle

Écrivez une expression lambda implémentant `Predicate<Integer>` qui teste si un nombre est pair, puis utilisez-la pour filtrer une liste `List<Integer> nombres` avec un flux (stream).

#### Correction

```
Predicate<Integer> estPair = n -> n % 2 == 0;  
List<Integer> pairs = nombres.stream()  
    .filter(estPair)  
    .collect(Collectors.toList());
```



# Chapitre 6 — Gestion d'erreurs et immutabilité

## 6.1 Exceptions personnalisées

Créer ses propres exceptions rend le code plus expressif et permet de distinguer précisément les erreurs métier des erreurs techniques.

```
class SoldeInsuffisantException extends Exception {
    public SoldeInsuffisantException(String message) { super(message); }
}

void retirer(double montant) throws SoldeInsuffisantException {
    if (montant > solde)
        throw new SoldeInsuffisantException("Solde insuffisant");
    solde -= montant;
}
```

**Checked vs unchecked** : une exception `Exception` (checked) doit être déclarée avec `throws` ou capturée ; une exception `RuntimeException` (unchecked) n'y est pas obligée. On réserve généralement les checked exceptions aux erreurs récupérables que l'appelant doit gérer.

## 6.2 Immutabilité

Un objet **immuable** ne peut pas changer d'état après sa création. Cela simplifie énormément le raisonnement sur le code, en particulier en contexte concurrent (pas besoin de synchronisation pour un objet qui ne change jamais).

```
final class Point {
    private final double x, y;
    public Point(double x, double y) { this.x = x; this.y = y; }
    public double getX() { return x; }
    public Point deplacer(double dx, double dy) {
        return new Point(x + dx, y + dy); // nouvel objet, pas de mutation
    }
}
```

- Classe déclarée `final` (pas de sous-classe qui casserait l'immutabilité).
- Tous les champs `private final`, initialisés uniquement dans le constructeur.
- Aucun setter ; toute « modification » renvoie un nouvel objet.

### Exercice 5 — Rendre une classe immuable

Voici une classe mutable :

```
class CompteBancaire { private double solde; public void setSolde(double s) {
    solde = s; } }
```

Réécrivez-la pour qu'elle soit immuable, avec une méthode `avecDepot(double montant)` qui renvoie un nouveau compte.

## Correction

```
final class CompteBancaire {  
    private final double solde;  
    public CompteBancaire(double solde) { this.solde = solde; }  
    public double getSolde() { return solde; }  
    public CompteBancaire avecDepot(double montant) {  
        return new CompteBancaire(solde + montant);  
    }  
}
```

# Chapitre 7 — Exercices récapitulatifs

## Exercice 6 — Conception d'un mini-système

Vous concevez un système de notifications (email, SMS, push) pour une application.

- a) Quel pattern permettrait d'ajouter facilement un nouveau canal de notification sans modifier le code existant ?
- b) Écrivez l'interface correspondante et une implémentation pour l'email (signature seulement).
- c) Quel principe SOLID cette conception respecte-t-elle particulièrement ?

### Correction

a) Strategy (ou Factory pour la création des canaux) : chaque canal de notification devient une implémentation interchangeable d'une interface commune.

```
b) interface CanalNotification { void envoyer(String message, String destinataire);  
}  
class NotificationEmail implements CanalNotification {  
    public void envoyer(String message, String destinataire) { ... }  
}
```

c) Le principe Open/Closed (O) : on peut ajouter un nouveau canal (SMS, push) en créant une nouvelle classe, sans toucher au code qui orchestre l'envoi des notifications.

## Exercice 7 — Question ouverte

Expliquez en quelques phrases pourquoi l'immuabilité et le principe de responsabilité unique (S de SOLID) se renforcent mutuellement dans une architecture orientée objet.

### Correction

Un objet immuable ne peut être modifié qu'en créant un nouvel objet, ce qui pousse naturellement à isoler chaque transformation dans une méthode dédiée plutôt que de laisser plusieurs parties du code modifier librement l'état d'un même objet partagé. Cela réduit les raisons pour lesquelles une classe pourrait changer, ce qui va exactement dans le sens du principe de responsabilité unique. Ensemble, les deux principes rendent le code plus facile à tester et à raisonner, car l'état d'un objet à un instant donné ne dépend plus d'un historique complexe de mutations.